REACT/Next.js — MOST USED & MOST ASKED TOPICS

1️⃣ Core Fundamentals (100% asked)

These are **non-negotiable**.

- JSX (why not HTML?)
- Virtual DOM vs Real DOM
- Reconciliation & Diffing Algorithm
- Functional Components vs Class Components
- Props vs State
- One-way data flow
- Controlled vs Uncontrolled components
- Lifting state up
- Conditional rendering
- Lists & keys (why keys matter)
- Fragment (<> </>)
- Synthetic events

👉 **Interview focus**: *Why React is fast, how rendering works*

2️⃣ Hooks (VERY HOT 🔥)

Expect **deep questions**, not definitions.

Core Hooks

- useState
- useEffect
  - dependency array
  - cleanup
  - infinite loop traps

- useContext
- useRef
- useMemo
- useCallback

Advanced / Less Obvious

- useReducer (Redux alternative)
- useLayoutEffect
- useImperativeHandle
- Custom Hooks (VERY IMPORTANT)

👉 **Interview focus**:

> When to use useMemo vs useCallback

> Why useRef doesn't cause re-render

> Side effects vs render phase

3️⃣ Rendering & Performance (Frequently Asked)

This separates **junior vs senior**.

- Re-render causes
- React.memo
- Avoiding unnecessary re-renders
- Key-based reconciliation issues
- Expensive computation optimization
- Lazy loading (React.lazy, Suspense)
- Code splitting
- Debouncing & throttling
- Virtualized lists (large data)

👉 **Production focus**: dashboards, tables, ERP, analytics apps

---

4️⃣ State Management (VERY COMMON)

You **must** explain tradeoffs.

- Local state vs global state
- Context API (pros/cons)
- Redux (classic)
- Redux Toolkit
- RTK Query
- Zustand / Jotai (modern alternatives)
- When NOT to use Redux
- Normalized state
- Immutable updates

👉 **Interview focus**:

> Why Redux Toolkit exists

> Redux vs Context

> Global state anti-patterns

5️⃣ Component Design & Architecture

Very important for **clean code** people like you.

- Presentational vs Container components
- Smart vs Dumb components
- Reusable components
- Compound components pattern
- Render props
- Higher Order Components (HOC)
- Controlled component patterns
- Separation of concerns

👉 **Architect mindset** topic 🧠

6️⃣ Forms & User Input (Always Asked)

- Controlled forms
- Validation strategies
- React Hook Form
- Formik
- Custom validation
- Handling large forms
- Error handling UX

7️⃣ Routing (React Side)

- React Router basics
- Dynamic routes
- Nested routes
- Protected routes
- Lazy loaded routes
- URL params vs query params

8️⃣ API & Data Fetching

- Fetch vs Axios
- Error handling
- Loading states
- Pagination
- Infinite scroll
- Caching strategies
- Retry logic
- Optimistic updates

9️⃣ Styling in React

- CSS Modules
- Tailwind CSS
- Styled Components
- Inline styles (why bad)
- Theme handling
- Dark mode
- Responsive UI

🔟 Testing (Often Ignored, But Asked)

- Unit vs Integration tests
- Jest
- React Testing Library
- Mocking API calls
- Testing hooks
- Testing forms

⚡ NEXT.JS — MOST USED & MOST ASKED TOPICS (COMPLETE)

1️⃣ Next.js Fundamentals (MUST KNOW)

- Why Next.js over React?
- File-based routing
- App Router vs Pages Router
- Layouts
- Metadata & SEO
- Image optimization
- Font optimization

2️⃣ Rendering Strategies (🔥 VERY HOT 🔥)

This is where interviews get **serious**.

- CSR (Client Side Rendering)
- SSR (Server Side Rendering)
- SSG (Static Site Generation)
- ISR (Incremental Static Regeneration)
- Streaming & Suspense
- When to use which (VERY IMPORTANT)

👉 **Interview gold question**:

> ERP dashboard vs Blog vs Landing page — which rendering?

3️⃣ Data Fetching in Next.js

- fetch() in server components
- Caching & revalidation
- Server Actions
- Parallel data fetching
- Waterfall problem
- API routes vs server actions

4️⃣ App Router Deep Dive (Next 13+)

- Server Components vs Client Components
- "use client" directive
- Component boundaries
- Shared layouts
- Loading & error files
- Route groups
- Middleware

5️⃣ Authentication & Authorization

- JWT vs Session
- Cookies vs LocalStorage
- NextAuth
- Middleware-based auth
- Role-based access
- Protected routes

6️⃣ Performance & Optimization

- Image optimization
- Dynamic imports
- Edge functions
- Caching strategies
- Reducing JS bundle size
- Prefetching routes

7️⃣ SEO & Metadata (VERY IMPORTANT)

- Meta tags
- Open Graph
- Structured data
- Server-rendered SEO
- Dynamic metadata
- Sitemap & robots

8️⃣ API Routes & Backend Use

- API routes vs external backend
- Route handlers
- REST vs server actions

- Security concerns
- Rate limiting

9️⃣ Deployment & Production

- Vercel deployment
- Environment variables
- Build vs runtime configs
- CI/CD basics
- Monitoring & logs

🔟 Common Pitfalls (Interview Favorites)

- Hydration mismatch
- Using browser APIs in server components
- Overusing client components
- Auth leaks
- SEO mistakes
- Cache bugs

🧠 SYSTEM DESIGN / ARCHITECT LEVEL QUESTIONS (YOU SHOULD MASTER)

- How would you design a large ERP frontend?
- How state flows across pages?
- How auth works across SSR & CSR?
- How to handle permissions?
- How to optimize large tables?
- How to scale frontend architecture?
- How to secure frontend?

🎯 HOW TO USE THIS LIST (SMART WAY)

Since you want **accuracy + architecture mindset**, I suggest:

- ✅ Revise **Core + Hooks + Rendering** first
- ✅ Then **Next.js rendering & data fetching**
- ✅ Then **state management + auth**
- ✅ Finally **performance + architecture**